



青岛科技大学
Qingdao University of Science & Technology

数据结构与算法分析2

第7章 动态规划1—基本原理

梁宏涛

lht@qust.edu.cn

周二&周四上午

1/56

提纲
CONTENTS

第7章—动态规划

7.1 动态规划概述

7.2 最大连续子序列和

7.8*完全背包&多重背包问题

7.3 最长递增子序列

7.9扔鸡蛋问题

7.4 三角形最小路径

7.10资源分配问题

7.5 最长公共子序列

7.11旅行商问题

7.6 编辑距离

7.12最少士兵数问题

7.7 0/1背包问题

7.13矩阵连乘问题

2/56

提纲
CONTENTS

第7章—动态规划

7.1 动态规划概述

7.2 最大连续子序列和

7.8*完全背包&多重背包问题

7.3 最长递增子序列

7.9扔鸡蛋问题

7.4 三角形最小路径

7.10资源分配问题

7.5 最长公共子序列

7.11旅行商问题

7.6 编辑距离

7.12最少士兵数问题

7.7 0/1背包问题

7.13矩阵连乘问题

3/56

7.1 动态规划概述

7.1.1 从一个简单示例入门

【例7.1】分析例2.3求斐波那契数列递归算法的缺点，改进其时间性能。

【例2.3】斐波那契数列定义为：

$\text{Fib}(n)=1$	当 $n=1$ 时
$\text{Fib}(n)=1$	当 $n=2$ 时
$\text{Fib}(n)=\text{Fib}(n-1)+\text{Fib}(n-2)$	当 $n>2$ 时

对应的递归算法如下：

```
int Fib(int n) {                                //递归求Fibonacci数列
    if (n==1 || n==2)
        return 1;
    else
        return Fib(n-1)+Fib(n-2);
}
```

画出求**Fib(5)**的递归树。

4/56

主观题 10分

【递归】斐波那契数列定义为：

$\text{Fib}(n)=1$

当 $n=1$ 时

$\text{Fib}(n)=1$

当 $n=2$ 时

$\text{Fib}(n)=\text{Fib}(n-1)+\text{Fib}(n-2)$

当 $n>2$ 时

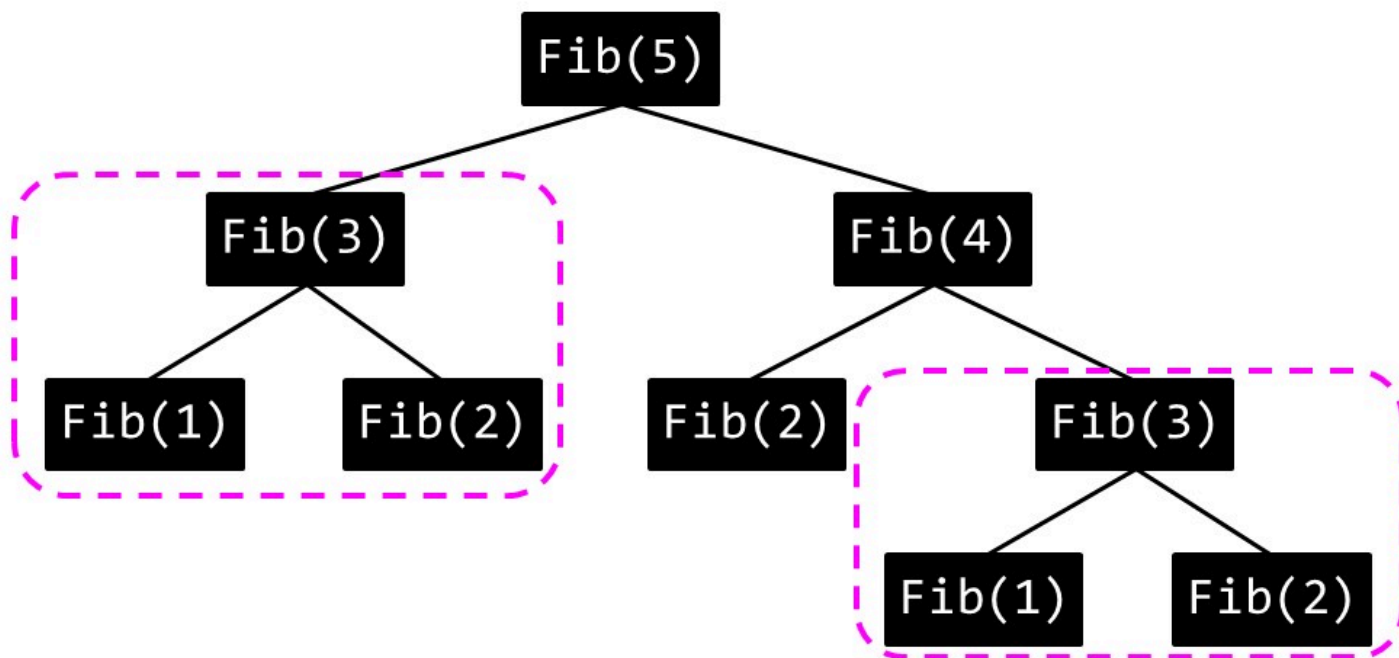
对应的递归算法如下：

```
int Fib(int n) { // 递归求 Fibonacci 数列
    if (n==1 || n==2)
        return 1;
    else
        return Fib(n-1)+Fib(n-2);
}
```

画出求 $\text{Fib}(5)$ 的递归树。

解

- **Fib(n)** 算法非常低效。
- 将问题 **Fib(n)** 转换为子问题 **Fib($n-2$)** 和 **Fib($n-1$)**, 然后单独求解这两个子问题, 实际上求解 **Fib($n-1$)** 中又包含求解 **Fib($n-2$)**
- 也就是说求解 **Fib(n)** 中存在大量重复的子问题。



6/56



如何避免重叠子问题的重复计算呢？

- 设计一个一维 dp 数组，用 $dp[i]$ 存放 $Fib(i)$ 的值。
- 首先将 dp 所有元素置为 0 ，一旦求出 $Fib(i)$ 就将其结果保存在 $dp[i]$ 中（此时 $dp[i] > 0$ ）。
- 在计算 $Fib(i)$ 时先查看 $dp[i]$ ，若 $dp[i] \neq 0$ 说明 $Fib(i)$ 是一个重叠子问题，前面已经求出结果，此时只需要返回 $dp[i]$ 即可。

```
1. int dp[MAXN]; //全局变量

2. int Fib1(int n) { //递归算法:被Fib1调用
3.     if(dp[n]!=0) return dp[n];
4.     if(n==1) dp[n]=1;
5.     else if(n==2) dp[n]=1;
6.     else dp[i]=Fib1(n-2)+Fib1(n-1);
7.     return dp[n];
8. }

9. int Fib(int n) { //求斐波那契数列
10.     memset(dp,0,sizeof(dp)); //初始化元素为0
11.     return Fib1(n);
12. }
```

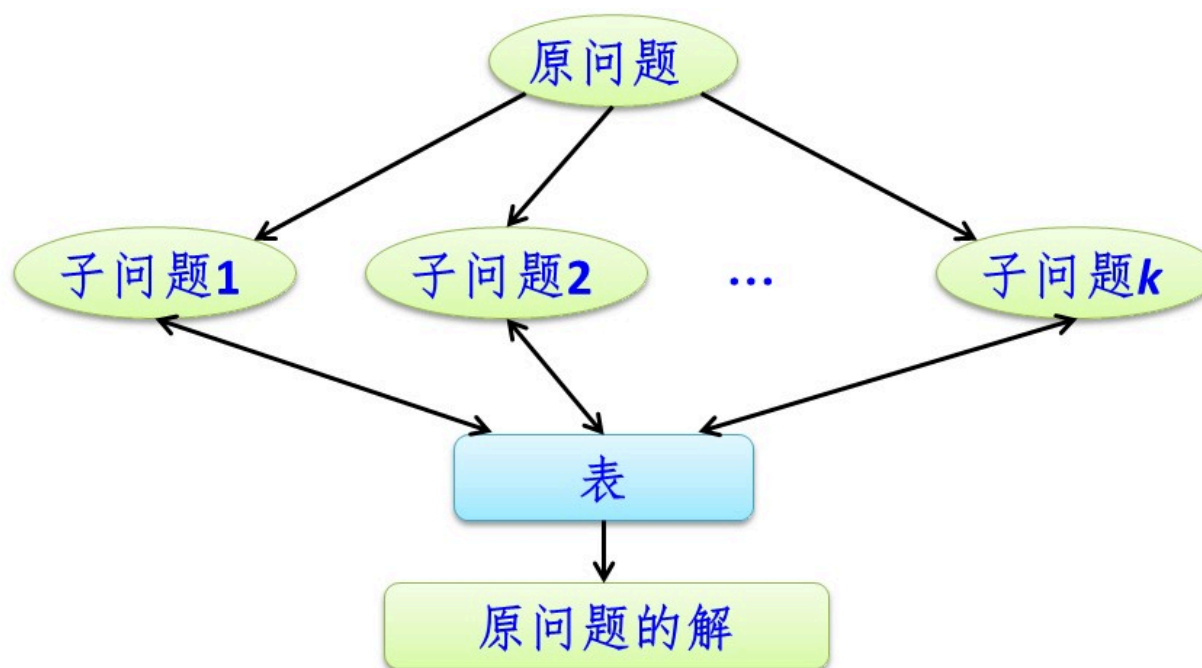

对应的迭代算法

```
1. int dp[MAXN]; //全局变量
2. dp[1]=1; dp[2]=1;

3. int Fib2(int n) { //迭代算法:求斐波那契数列
4.     if (dp[n] != 0) return dp[n];
5.     else return Fib2(n-1) + Fib2(n-2);
6. }

7. int Fib(int n) { //求斐波那契数列
8.     memset(dp,0,sizeof(dp)); //初始化元素为0
9.     return Fib2(n);
10. }
```

- 上述 $\text{Fib2}(n)$ 属于动态规划算法，其中数组 dp （表）称为动态规划数组。
- 动态规划就是记录子问题的结果再利用的方法。求大问题解中遇到子问题时直接查表。



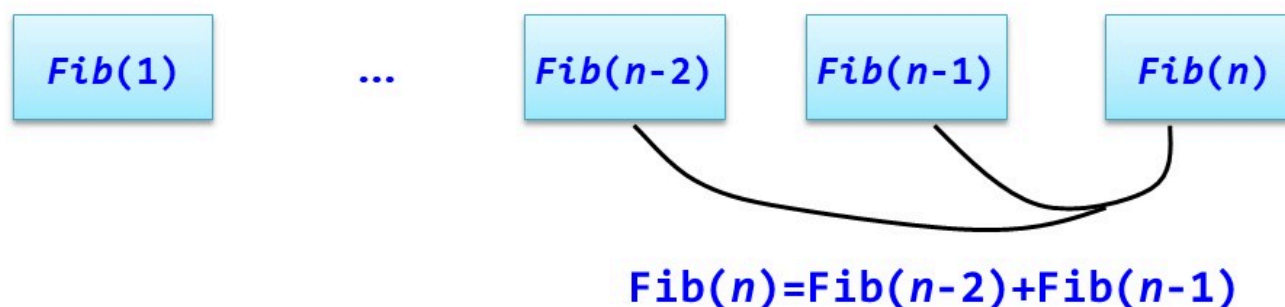
■ 动态规划求解过程为自底向上，由子问题解产生大问题解。

■ 相对应Fib1(n)算法是动态规划的变形，称为备忘录方法。
采用递归实现：自顶向下。

■ 动态规划算法中动态规划数组dp用于存放子问题解，一般是存放连续的解。

■ 可以对dp的下标进行特殊处理，使每次操作仅保留若干有用信息，新的元素不断循环刷新，这样数组的空间被滚动地利用，称为滚动数组。

滚动数组:



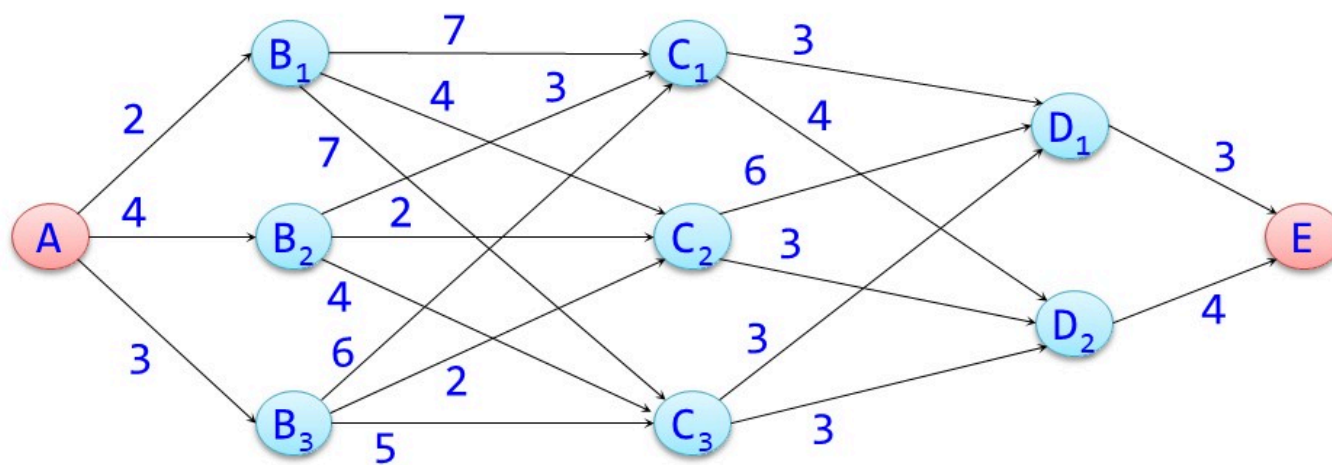
$Fib(i)$ 的值存放在 $dp[(i-1)\%3]$ 中

```
1. int Fib3(int n) { //求斐波那契数列
2.     int dp[3];
3.     dp[0]=1; dp[1]=1;
4.     for (int i=2;i<n;i++)
5.         dp[i%3]=dp[(i-1)%3]+dp[(i-2)%3];
6.     return dp[(n-1)%3];
7. }
```


7.1.2 动态规划原理

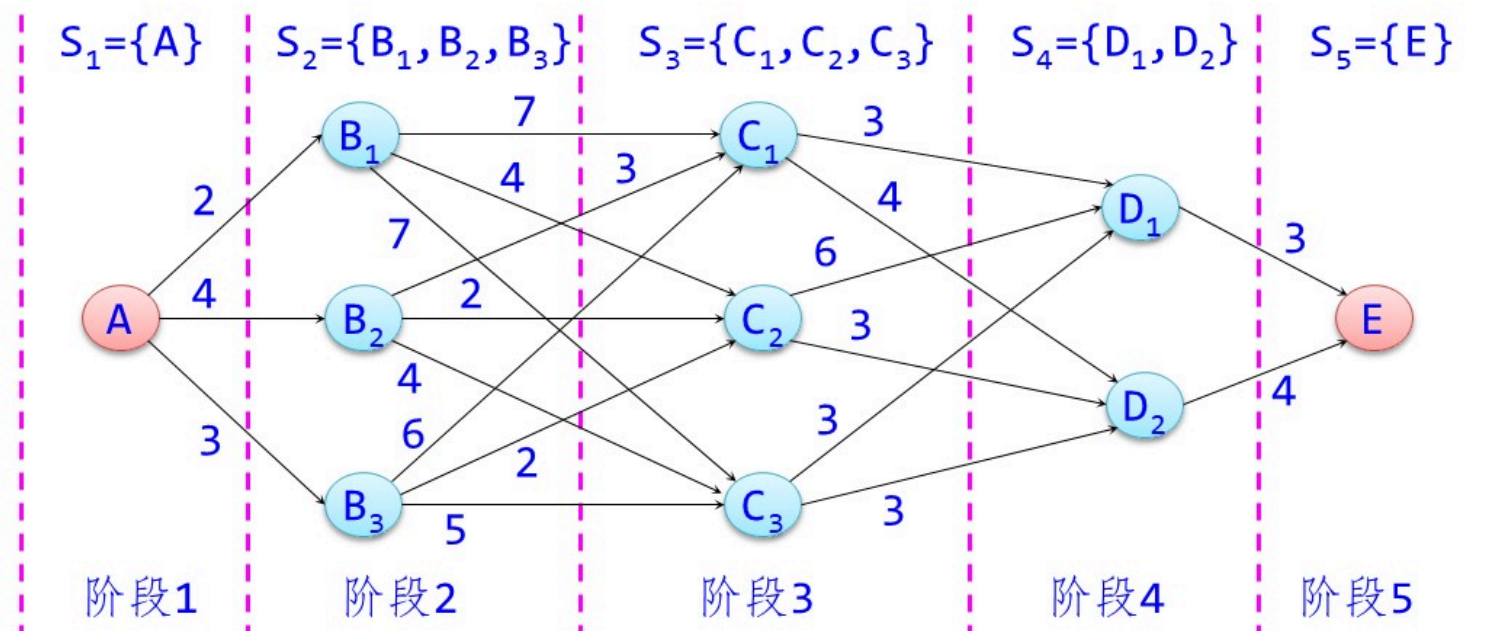
1. 动态规划的相关概念

- 一个多段图 $G=(V, E)$ ，在顶点A处有一水库，现需要从顶点A铺设一条管道到顶点E，边上的数字表示对应两个顶点之间的距离，该图采用邻接矩阵A表示。
- 现要找出一条从顶点A到顶点E的线路，使得铺设的管道长度最短。



13/56

- 分为5个阶段，通常阶段变量用 k 表示，这里 k 为1~5。阶段 k 可能有多个状态，通常用状态集合 S_k 表示，例如 $S_2=\{B_1, B_2, B_3\}$ 。
- 状态变量 x_k 表示 S_k 中某个状态，如 x_2 可以取 S_2 中的任意值。



设最优指标函数 $f(s)$ 表示状态 s 到终点 E 的最短路径长度，用 k 表示阶段，对应的状态转移方程如下：

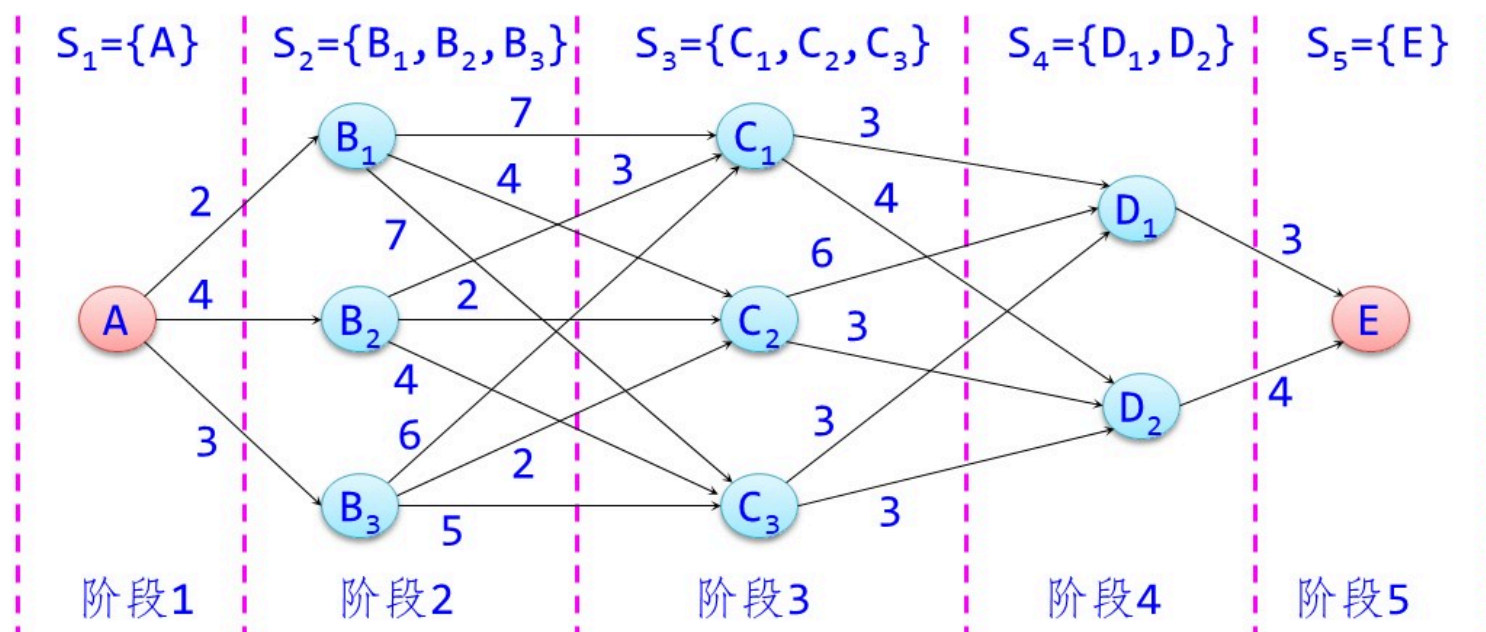
$$f_5(E)=0$$

$$f_k(s_k) = \min_{x_k \in D_k(s_k)} \{c(s_k, x_k) + f_{k+1}(s_{k+1})\}$$

或者简写为：

$$f(E)=0$$

$$f(s) = \min_{\langle s, t \rangle \in E} \{c(s, t) + f(t)\}$$

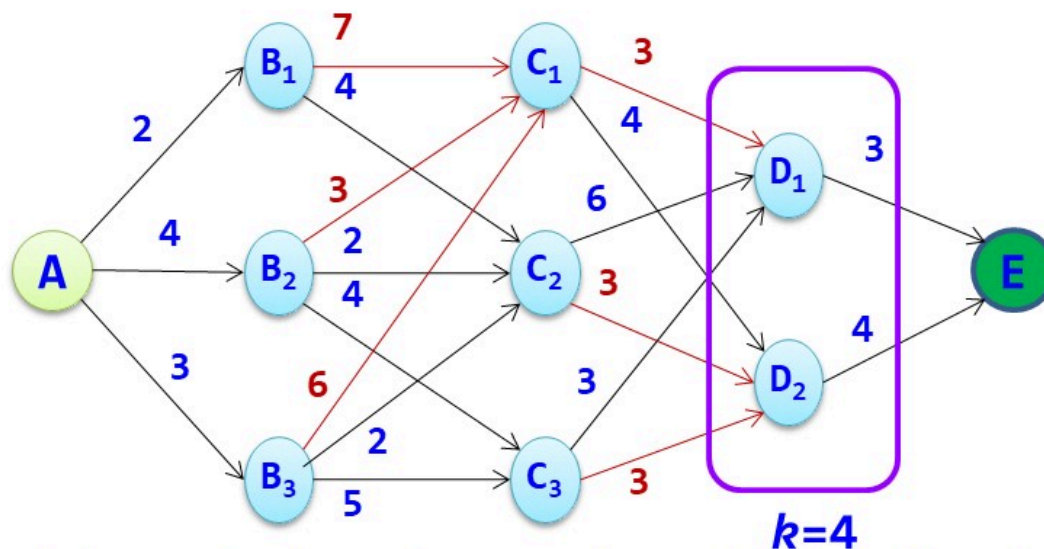


15/56

2. 动态规划问题的解法

- 逆序解法
- 顺序解法

(1) 动态规划问题的逆序解法



$f(s)$ 表示状态 s 到终点 E 的最短路径长度

求解 $E \rightarrow A$ 的过程（用 $next$ 表示路径上一个顶点的后继顶点）：

① 第5阶段

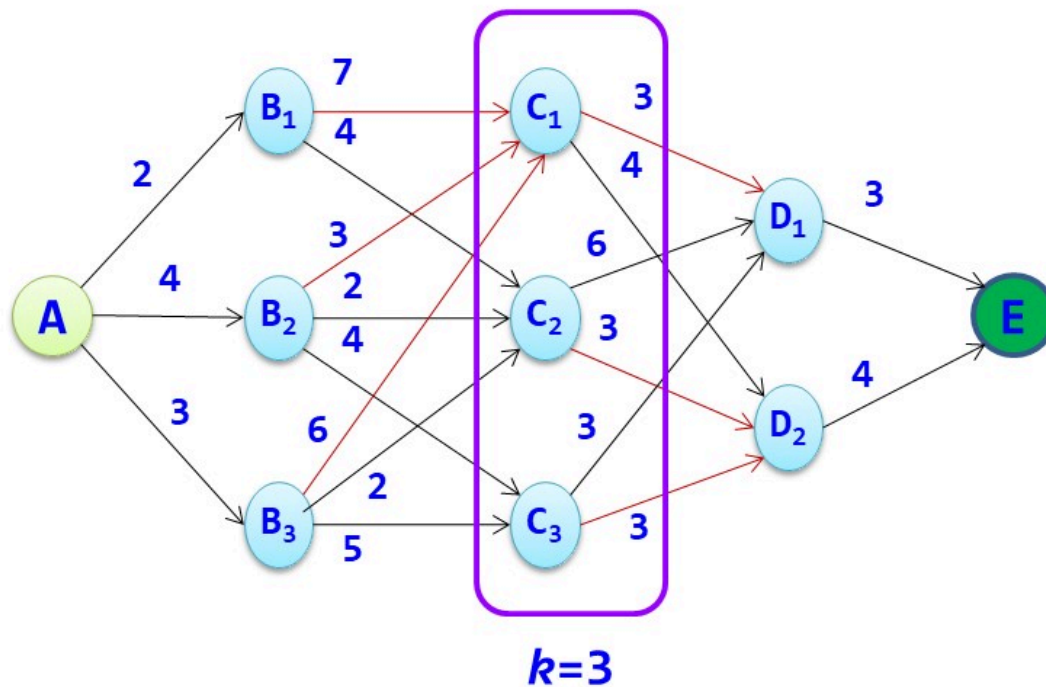
$$f(E)=0$$

② 第4阶段

$$f(D_1)=\text{MIN}(c(D_1, E)+f(E))=3, \quad \text{next}(D_1)=E$$

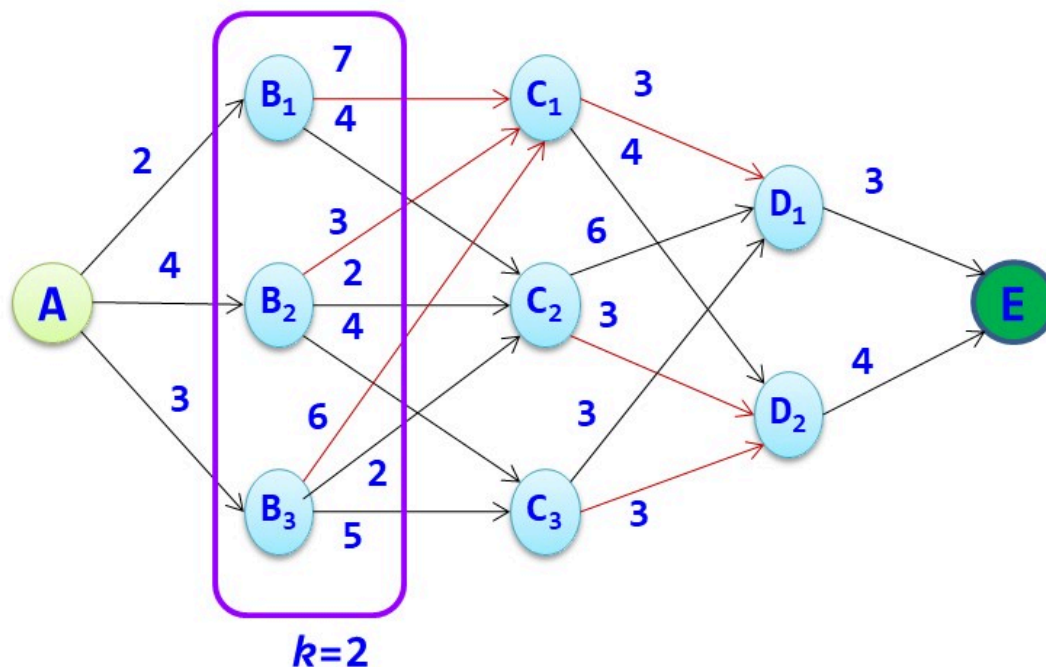
$$f(D_2)=\text{MIN}(c(D_2, E)+f(E))=4, \quad \text{next}(D_2)=E$$

17/56



③ 第3阶段

$$\begin{aligned}
 f(C_1) &= \min \begin{pmatrix} c(C_1, D_1) + f(D_1) = 6 \\ c(C_1, D_2) + f(D_2) = 8 \end{pmatrix} = 6, \text{ next}(C_1)=D_1 \\
 f(C_2) &= \min \begin{pmatrix} c(C_2, D_1) + f(D_1) = 9 \\ c(C_2, D_2) + f(D_2) = 7 \end{pmatrix} = 7, \text{ next}(C_2)=D_2 \\
 f(C_3) &= \min \begin{pmatrix} c(C_3, D_1) + f(D_1) = 6 \\ c(C_3, D_2) + f(D_2) = 7 \end{pmatrix} = 6, \text{ next}(C_3)=D_1
 \end{aligned}$$

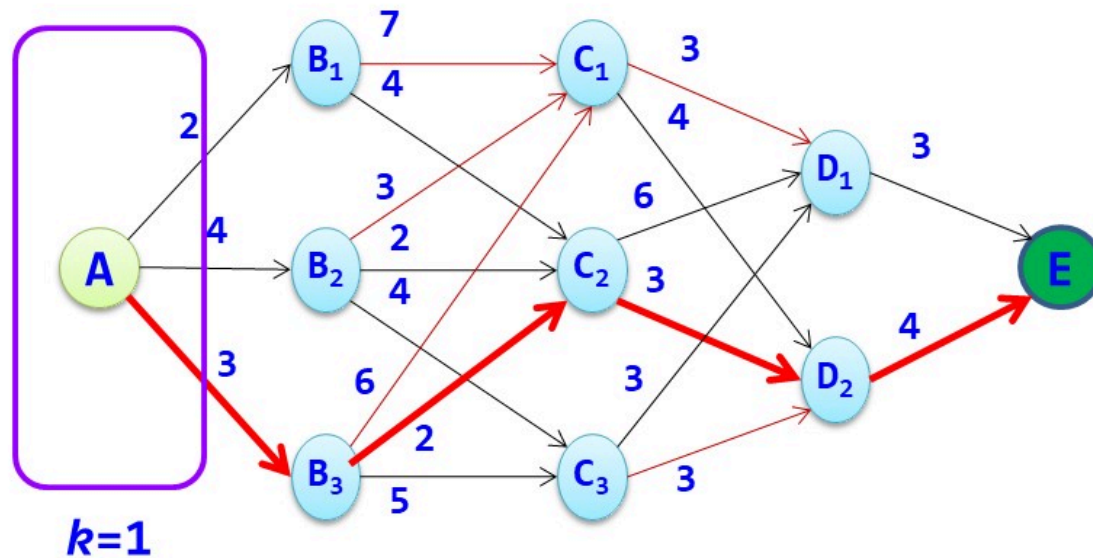


④ 第2阶段

$$f(B_1) = \min \begin{pmatrix} c(B_1, C_1) + f(C_1) = 13 \\ c(B_1, C_2) + f(C_2) = 11 \\ c(B_1, C_3) + f(C_3) = \infty \end{pmatrix} = 11, \text{ next}(B_1) = C_2$$

$$f(B_2) = \min \begin{pmatrix} c(B_2, C_1) + f(C_1) = 9 \\ c(B_2, C_2) + f(C_2) = 9 \\ c(B_2, C_3) + f(C_3) = 10 \end{pmatrix} = 9, \text{ next}(B_2) = C_1$$

$$f(B_3) = \min \begin{pmatrix} c(B_3, C_1) + f(C_1) = 12 \\ c(B_3, C_2) + f(C_2) = 9 \\ c(B_3, C_3) + f(C_3) = 11 \end{pmatrix} = 9, \text{ next}(B_3) = C_2$$



⑤ 第1阶段

$$f(A) = \text{MIN} \begin{pmatrix} c(A, B_1) + f(B_1) = 13 \\ c(A, B_2) + f(B_2) = 13 \\ c(A, B_3) + f(B_3) = 12 \end{pmatrix} = 12, \text{ next}(A) = B_3$$

由 $f(A)=12$ 求出的最短路径长度为12

由 $\text{next}(A)=B_3$, $\text{next}(B_3)=C_2$, $\text{next}(C_2)=D_2$, $\text{next}(D_2)=E$, 推出最短路径为 $A \rightarrow B_3 \rightarrow C_2 \rightarrow D_2 \rightarrow E$ 。

20/56

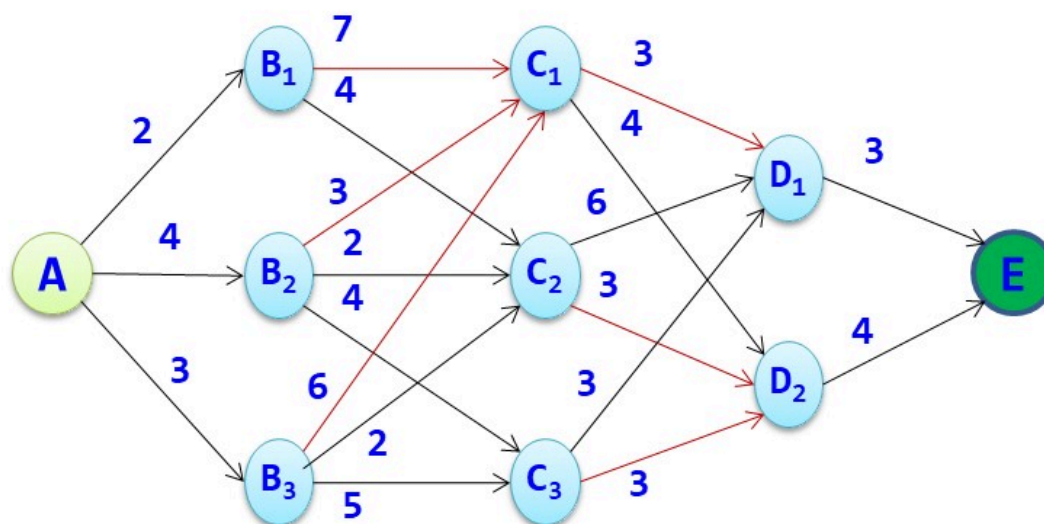
(2) 动态规划问题的顺序解法

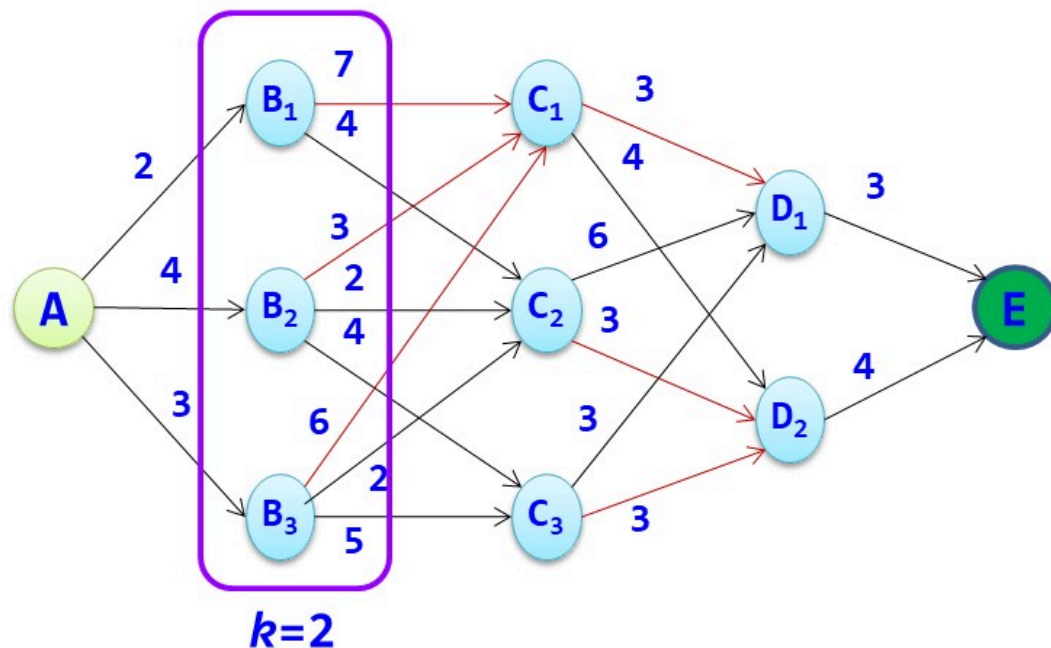
$A \rightarrow E$ 对应的的状态转移方程如下：

$$f(A)=0$$

$$f(t) = \min_{\langle s, t \rangle \in E} \{f(s) + c(s, t)\}$$

$f(t)$ 表示初始状态**A**到
状态**t**的最短路径长度





$f(s)$ 表示初始状态A到状态s的最短路径长度

用pre表示路径上一个顶点的前驱顶点，其求解A→E的过程：

① 第1阶段

$$f(A)=0$$

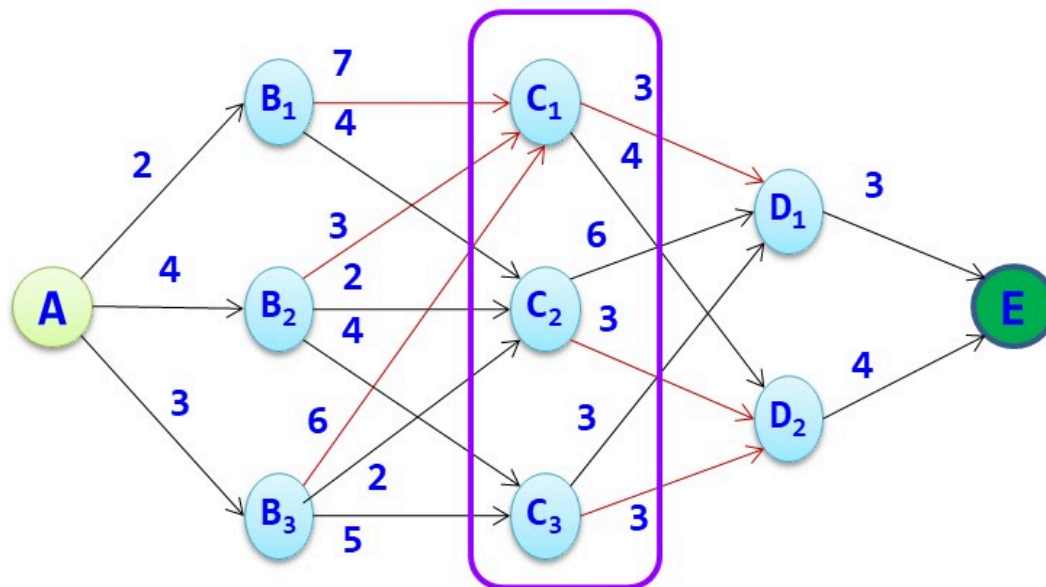
② 第2阶段

$$f(B_1)=\text{MIN}(f(A)+c(A, B_1))=2, \quad \text{pre}(B_1)=A$$

$$f(B_2)=\text{MIN}(f(A)+c(A, B_2))=4, \quad \text{pre}(B_2)=A$$

$$f(B_3)=\text{MIN}(f(A)+c(A, B_3))=3, \quad \text{pre}(B_3)=A$$

22/56



③ 第3阶段

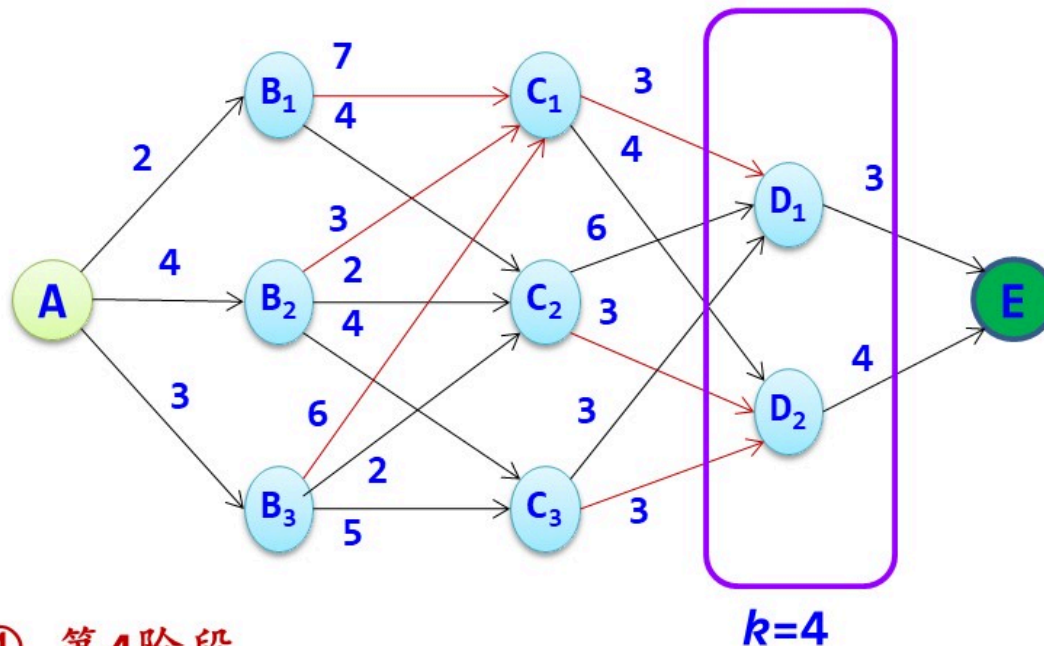
$k=3$

$$f(C_1) = \min \begin{pmatrix} f(B_1) + c(B_1, C_1) = 9 \\ f(B_2) + c(B_2, C_1) = 7 \\ f(B_3) + c(B_3, C_1) = 9 \end{pmatrix} = 7, \text{ pre}(C_1) = B_2$$

$$f(C_2) = \min \begin{pmatrix} f(B_1) + c(B_1, C_2) = 6 \\ f(B_2) + c(B_2, C_2) = 6 \\ f(B_3) + c(B_3, C_2) = 5 \end{pmatrix} = 5, \text{ pre}(C_2) = B_3$$

$$f(C_3) = \min \begin{pmatrix} f(B_1) + c(B_1, C_3) = \infty \\ f(B_2) + c(B_2, C_3) = 8 \\ f(B_3) + c(B_3, C_3) = 8 \end{pmatrix} = 8, \text{ pre}(C_3) = B_2$$

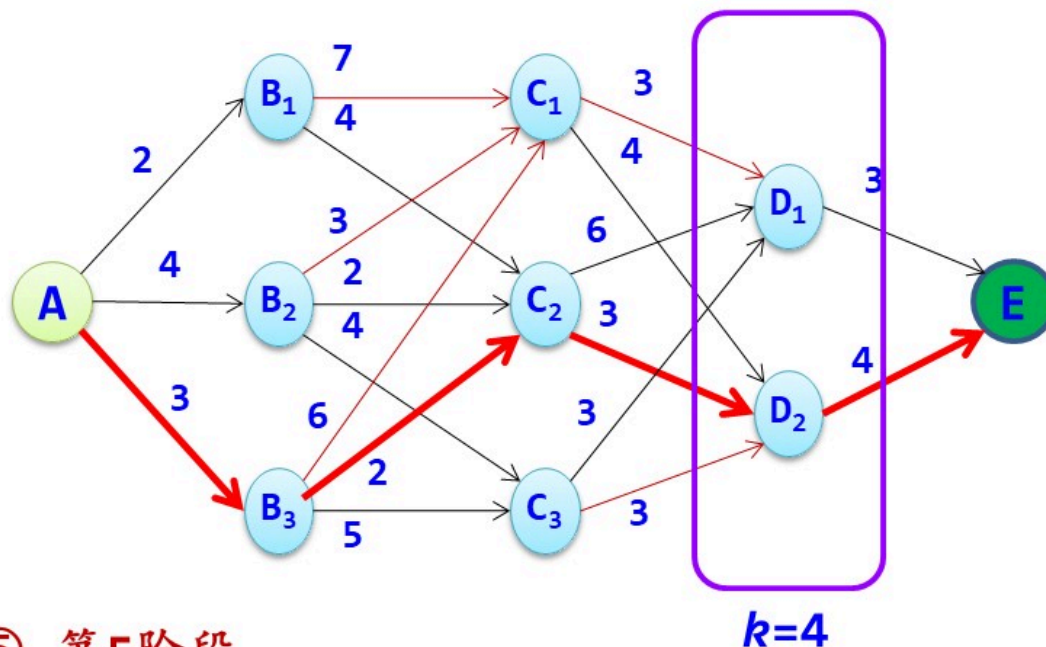
23/56



④ 第4阶段

$$f(D_1) = \min \begin{pmatrix} f(C_1) + c(C_1, D_1) = 10 \\ f(C_2) + c(C_2, D_1) = 11 \\ f(C_3) + c(C_3, D_1) = 11 \end{pmatrix} = 10, \text{ pre}(D_1) = C_1$$

$$f(D_2) = \min \begin{pmatrix} f(C_1) + c(C_1, D_2) = 11 \\ f(C_2) + c(C_2, D_2) = 8 \\ f(C_3) + c(C_3, D_2) = 11 \end{pmatrix} = 8, \text{ pre}(D_2) = C_2$$



⑤ 第5阶段

$$f(E) = \min \left(\begin{array}{l} f(D_1) + c(D_1, E) = 13 \\ f(D_2) + c(D_2, E) = 12 \end{array} \right) = 12, \text{ pre}(E) = D_2$$

由 $f(E)=12$ 求出的最短路径长度为12

由 $\text{pre}(E)=D_2$, $\text{pre}(D_2)=C_2$, $\text{pre}(C_2)=B_3$, $\text{pre}(B_3)=A$, 推出最短路径为 $A \rightarrow B_3 \rightarrow C_2 \rightarrow D_2 \rightarrow E$ 。

7.1.3 动态规划求解问题的类型、性质和步骤

1. 动态规划求解问题的类型

- 求目标函数指定的最大值或最小值。
- 判断某个条件是否可行。
- 统计满足某个条件的方案数。

2. 动态规划求解问题的具有的性质

(1) 最优子结构性质

- 如果问题的最优解所包含的子问题的解也是最优的，就称该问题具有最优子结构性质。

示例

$$f_5(E)=0$$

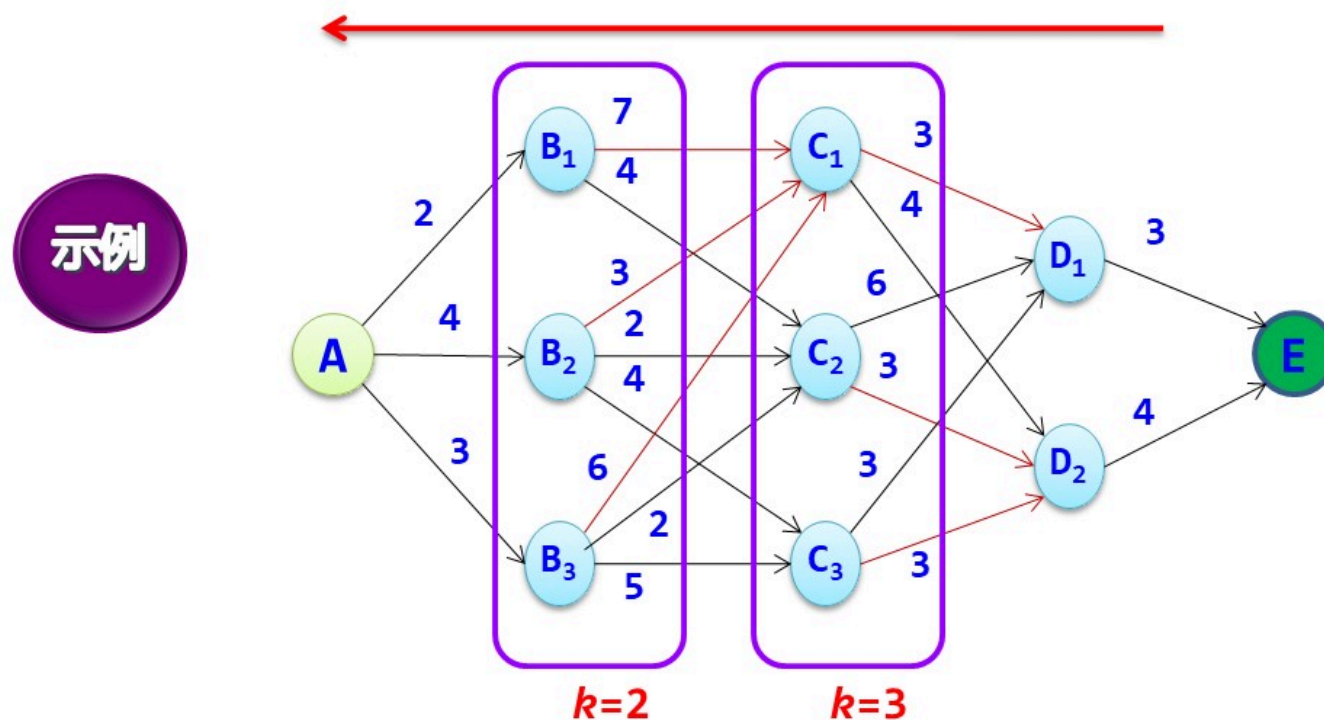
$$f_k(s_k) = \min_{x_k \in D_k(s_k)} \{c(s_k, x_k) + f_{k+1}(s_{k+1})\}$$

- 通常采用反证法来证明一个问题具有最优子结构性质。先假设由问题的最优解导出的子问题的解不是最优的，然后在这个假设下可构造出比原问题最优解更好的解，从而导致矛盾。

27/56

(2) 无后效性

■ 无后效性指的是这样一种性质：某阶段的状态一旦确定，则此后过程的演变不再受此前各状态及决策的影响。



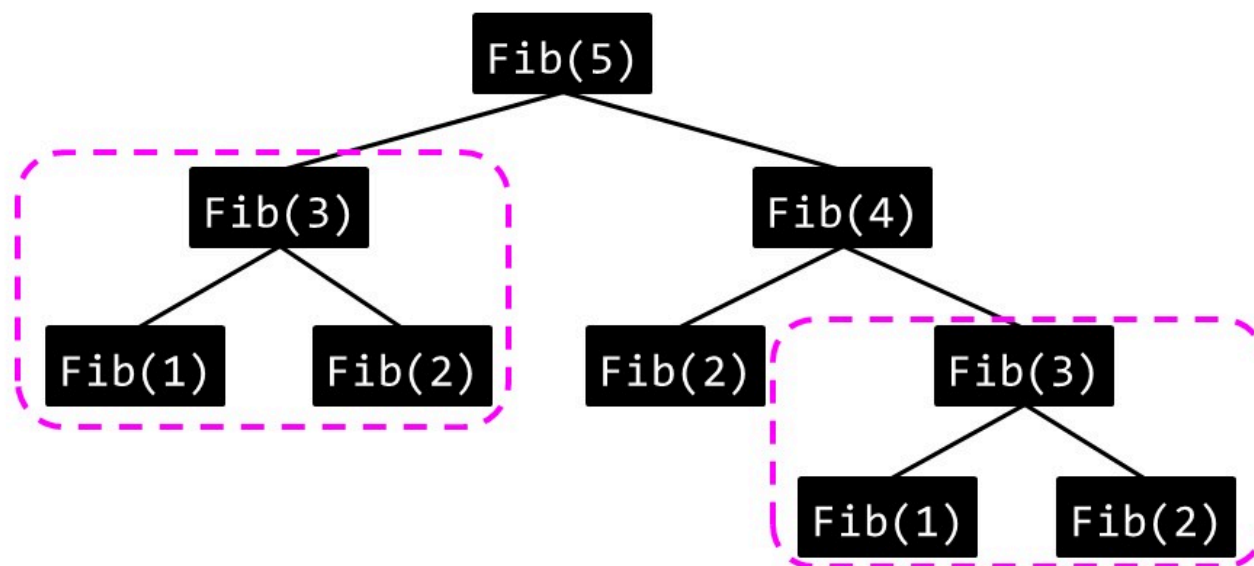
$k=2$ 阶段仅仅与 $k=3$ 阶段相关

28/56

(3) 重叠子问题

- 指一个问题分解的若干子问题之间是**不独立**的，其中一些子问题在后面的决策中可能被多次重复使用。

示例



29/56

3. 动态规划求解问题的步骤

- ① **确定状态**：将问题求解中各个阶段所处的各种情况用不同的状态表示出来。
- ② **确定状态转移方程**：描述求解中各个阶段的状态转移和指标函数的关系。
- ③ **确定初始条件和边界情况**：状态转移方程通常是一个递推式，初始条件通常指定递推的起点，在递推中需要考虑一些特殊情况，称为边界情况。
- ④ **确定计算顺序**：也就是指定求状态转移方程的顺序，是顺序求解还是逆序求解。
- ⑤ **消除冗余**：如采用滚动数组进一步提高时空性能。

提纲
CONTENTS

第7章—动态规划

7.1 动态规划概述

7.2 最大连续子序列和

7.8*完全背包&多重背包问题

7.3 最长递增子序列

7.9扔鸡蛋问题

7.4 三角形最小路径

7.10资源分配问题

7.5 最长公共子序列

7.11旅行商问题

7.6 编辑距离

7.12最少士兵数问题

7.7 0/1背包问题

7.13矩阵连乘问题

31/56

7.2 最大连续子序列和

 **问题描述：** 给定一个含 n ($n \geq 1$) 个整数的序列，要求求出其中最大连续子序列的和。

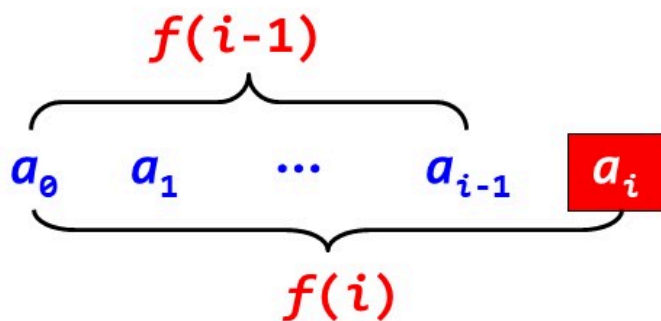
序列 $(-2, \underline{11, -4, 13}, -5, -2)$ 的最大子序列和为**20**。

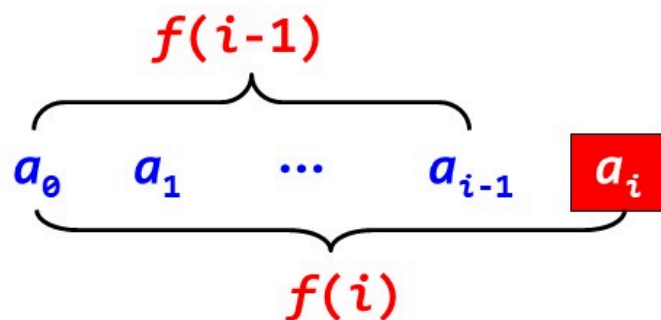
序列 $(-6, 2, 4, -7, \underline{5, 3, 2, -1, 6, -9, 10}, -2)$ 的最大子序列和为**16**。

规定一个序列最大连续子序列和至少是**0**，如果小于**0**，其结果为**0**。

解

- 先采用递归求解。
- 设 $f(i)$ 表示以元素 a_i 结尾的最大连续子序列和，为大问题。
- $f(i-1)$ 表示以元素 a_{i-1} 结尾的最大连续子序列和，为小问题。





考虑 a_i 的两种情况：

(1) 将 a_i 合并到前面以元素 a_{i-1} 结尾的最大连续子序列中，此时有 $f(i) = f(i-1) + a_i$ 。

(2) 不将 a_i 合并到前面以元素 a_{i-1} 结尾的最大连续子序列中，即以 a_i 结尾的最大连续子序列为 $\{a_i\}$ ，此时有 $f(i) = a_i$ 。



$$f(0) = a[0]$$

$$f(i) = \max(f(i-1) + a_i, a_i)$$

递归出口

当 $i > 0$ 时

$$f(0)=a[0]$$

递归出口

$$f(i)=\max(f(i-1)+a_i, a_i)$$

当 $i>0$ 时

- 最大连续子序列和 $\text{ans}=\max_{0 \leq i \leq n-1} f(i)$ 。
- 本题中最大连续子序列和至少为0（或者说最大连续子序列可以为空序列），所以最后的最大连续子序列和应该为 $\max(\text{ans}, 0)$ 。

- 如果采用递归算法实现，其中存在大量的重叠子问题，改为动态规划方法。
- 设置一维动态规划数组 dp ，用 $dp[i]$ 存放 $f(i)$ 的值，这样得到如下状态转移方程：

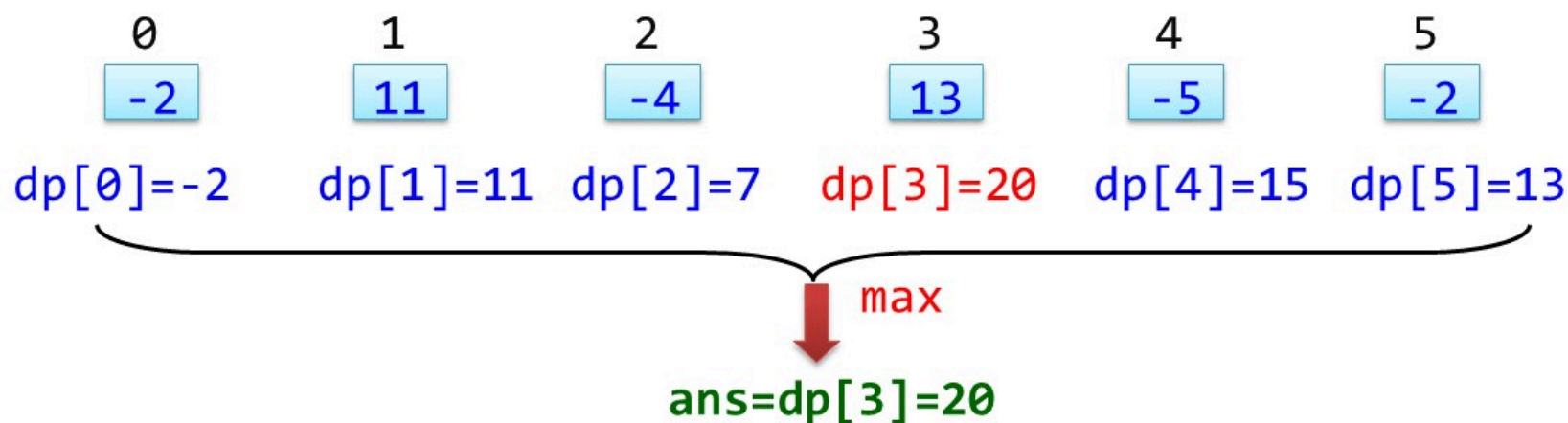
$$dp[0]=a[0]$$

初始条件

$$dp[i]=\max\{dp[i-1]+a_i, a_i\}$$

$i>0$

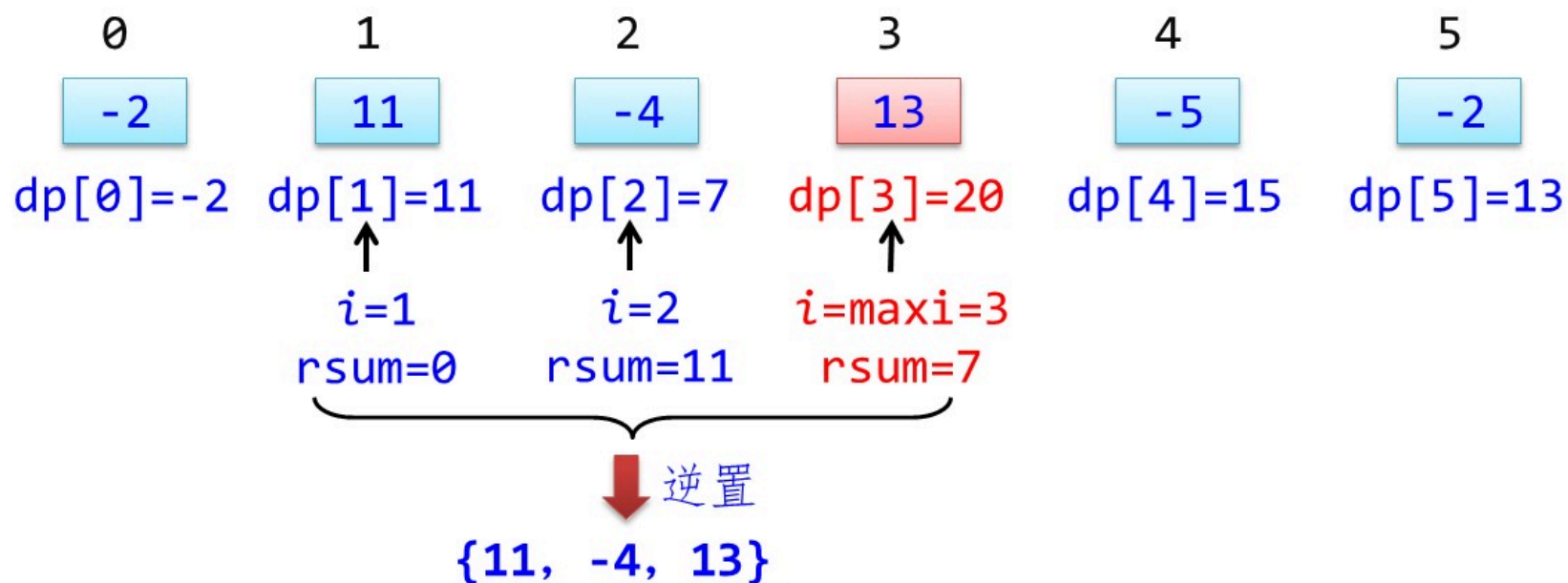
- 求出dp中的最大元素ans。
- 由于本题中最大连续子序列和至少为0（或者说最大连续子序列可以为空序列），所以最后的最大连续子序列和应该为 $\max(\text{ans}, 0)$ 。



1. <code>vector<int> dp;</code>	<code>//一维动态规划数组</code>
2. <code>int maxsubsum(int a[],int n) {</code>	<code>//求最大连续子序列和</code>
3. <code>dp=vector<int>(n,0);</code>	
4. <code>dp[0]=a[0];</code>	
5. <code>for(int i=1;i<n;i++) {</code>	
6. <code>dp[i]=max(dp[i-1]+a[i],a[i]);</code>	
7. <code>}</code>	
8.	
9. <code>int ans=dp[0];</code>	
10. <code>for(int i=1;i<n;i++)</code>	<code>//求最大dp元素ans</code>
11. <code>ans=max(ans,dp[i]);</code>	
12. <code>return max(ans,0);</code>	
13. <code>}</code>	

时间和空间复杂度: $O(n)$

- 当求出 dp 后可以推导出一个最大连续子序列（实际上这样的最大连续子序列可能有多，这里仅仅求出其中的一个）。
- 先在 dp 数组中求出最大元素的序号 $maxi$ ， i 从 $maxi$ 序号开始在 a 中向前查找， $rsum$ 从 $dp[maxi]$ 开始递减 $a[i]$ ，直到 $rsum$ 为0，对应的 a 中子序列就是一个最大连续子序列。



39/56

```
1. vector<int> maxsub(int a[],int n) { //求一个最大连续子序列
2.     vector<int> x;
3.     int maxi=0;
4.     for(int i=1;i<n;i++) {
5.         if(dp[i]>dp[maxi]) maxi=i;
6.     }
7.
8.     int rsum=dp[maxi];
9.     int i=maxi;
10.    while(i>=0 && rsum!=0) {
11.        rsum-=a[i];
12.        x.push_back(a[i]);
13.        i--;
14.    }
15.    reverse(x.begin(),x.end()); //逆置x
16.    return x;
17. }
```

40/56



【例7.2】连续子数组求和（LintCode402★★）。给定一个整数数组**A**，求出一个连续子数组（至少含一个元素），使得该子数组的和最大，输出答案时，请分别返回第一个数字和最后一个数字的下标，如果存在多个答案，请返回字典序最小的。

例如，**A={0, 1, 0, 1}**，答案为**{0, 3}**。

要求设计如下成员函数：

```
vector<int> continuousSubarraySum(vector<int> &A) {}
```

解

- 采用上述 $\text{maxsubsum}+\text{maxsub}$ 算法的思路。
- 用 maxi 和 maxj 表示最大子数组的第一个数字和最后一个数字的下标。

```
1. class Solution {
2.     vector<int> dp;                //一维动态规划数组
3. public:
4.     vector<int> continuousSubarraySum(vector<int> &A) {
5.         int n=A.size();
6.         if(n==1) return {0,0};
7.         dp=vector<int>(n,0);      //初始化数组dp
8.         dp[0]=A[0];
9.         int maxdp=A[0];
10.        int maxi,maxj=0;
11.        for(int i=1;i<n;i++) {
12.            dp[i]=max(dp[i-1]+A[i],A[i]);
13.            if(dp[i]>maxdp) {        //第一个最大dp元素
14.                maxdp=dp[i];
15.                maxj=i;
16.            }
17.        }
```

43/56

```
18.     int rsum=dp[maxj];
19.     maxi=maxj;
20.     while(maxi>=0 && (rsum!=0 || A[maxi]==0)) {
21.         rsum-=A[maxi];
22.         maxi--;
23.     }
24.     maxi++;
25.     return {maxi,maxj};
26. }
27. };
```



执行用时为**163ms**，内存消耗为**10.78MB**。

空间优化

- 如果只需要求最大连续子序列和，可以采用滚动数组优化空间。
- `maxSubSum`算法中用 `j` 标识阶段，由于 `dp[j]` 仅仅与 `dp[j-1]` 相关，将一维 `dp` 数组改为单个变量 `dp`。

```
1. int maxsubsum1(int a[],int n) { //求最大连续子序列和
2.     if(n==1) return a[0];
3.     int dp=a[0];
4.     int ans=dp;
5.     for(int j=1;j<n;j++) {
6.         dp=max(dp+a[j],a[j]);
7.         ans=max(ans,dp);
8.     }
9.     return max(ans,0);
10. }
```

空间复杂度为 $O(1)$

45/56

提纲
CONTENTS

第7章—动态规划

7.1 动态规划概述

7.2 最大连续子序列和

7.3 最长递增子序列

7.4 三角形最小路径

7.5 最长公共子序列

7.6 编辑距离

7.7 0/1背包问题

7.8*完全背包&多重背包问题

7.9扔鸡蛋问题

7.10资源分配问题

7.11旅行商问题

7.12最少士兵数问题

7.13矩阵连乘问题

46/56

7.3 最长递增子序列

 **问题描述：** 给定一个无序的整数序列 $a[0..n-1]$ ，求其中最长递增（严格）子序列的长度。

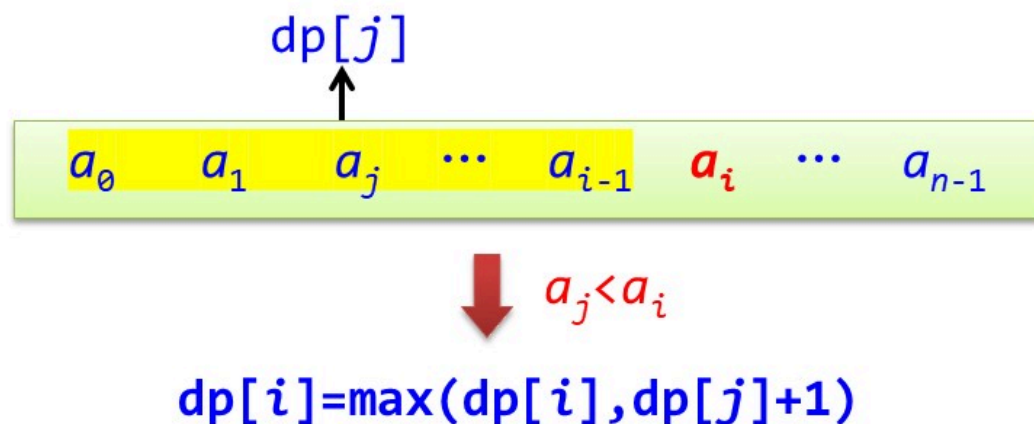
例如， $a=\{2, 1, 5, 3, 6, 4, 8, 9, 7\}$ ， $n=9$ ，其最长递增子序列为 $\{1, 3, 4, 8, 9\}$ ，结果为5。

解

- $dp[i]$ 表示以 $a[i]$ 结尾的最长递增子序列的长度。
- 计算顺序是 i 从 0 到 $n-1$ ，对于每个 $a[i]$ ， $dp[i]$ 置为 1 （表示只有 $a[i]$ 一个元素时最长递增子序列的长度为 1 ）。

考虑 $a[0..i-1]$ 中的每一个元素 $a[j]$ ，分为两种情况：

- ① 若 $a[j] < a[i]$ ，则以 a_j 结尾的最长递增子序列加上 a_i 可能构成一个更长的递增子序列，此时有 $dp[i] = \max(dp[i], dp[j] + 1)$ 。



- ② 否则最长递增子序列没有改变。

在求出 dp 数组后，通过顺序遍历 dp 求出最大值 ans ，则 ans 就是最长递增（严格）子序列的长度。

状态转移方程

$$dp[i]=1$$

$$0 \leq i \leq n-1 \text{ (初始条件)}$$

$$dp[i]=\max_{a[j]<a[i](j<i)}\{dp[j]+1\}$$

$$0 \leq i \leq n-1$$

```

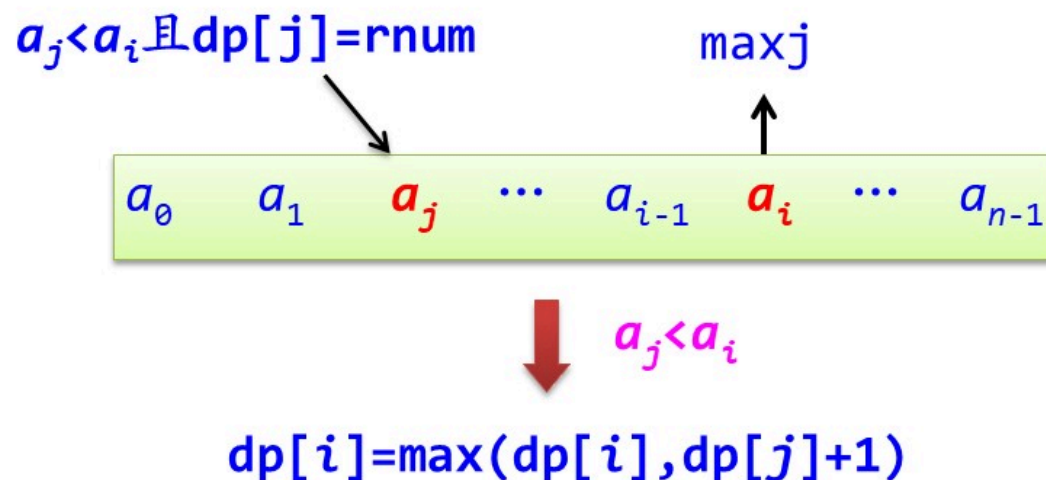
1. vector<int> dp; //一维动态规划数组
2. int maxinclen(int a[],int n) { //求最长递增子序列长度
3.     dp=vector<int>(n,0);
4.     for(int i=0;i<n;i++) {
5.         dp[i]=1;
6.         for(int j=0;j<i;j++) {
7.             if (a[i]>a[j]) dp[i]=max(dp[i],dp[j]+1);
8.         }
9.     }
10.
11. int ans=dp[0];
12.     for(int i=1;i<n;i++) //求dp中最大元素ans
13.         ans=max(ans,dp[i]);
14.     return ans;
15. }

```

【算法分析】 时间复杂度为 $O(n^2)$ ，空间复杂度为 $O(n)$ 。

dp $\Rightarrow$ x

- 当求出**dp**后可以推导出一个最长递增子序列。
- 先在**dp**数组中求出最大元素的序号**maxj**，从**a[maxj]**(含)向前查找**x**。
- 逆置**x**即可。



52/56

示例

	0	1	2	3	4	5	6	7	8		
$a=\{$		2	1	5	3	6	4	8	9	7	$\}$
$dp=\{$		1	1	2	2	3	3	4	5	4	$\}$
		↑	↑	↑	↑	↑	↑	↑			
		✓	×	✓	×	✓	✓	✓			

$x = \{9, 8, 4, 3, 1\}$



$x = \{1, 3, 4, 8, 9\}$

```
1. vector<int> maxincseq(int a[],int n) {  
2. //求一个最长递增子序列  
3.     vector<int> x;  
4.     int maxj=0;  
5.     for(int j=1;j<n;j++) {  
6.         if(dp[j]>dp[maxj]) maxj=j;    //求最大元素dp[minj]  
7.     }
```

```

8.   int rnum=dp[maxj];           //剩余的元素个数
9.   int j=maxj;                  //j指向当前最长递增子序列的一个元素
10.  x.push_back(a[j]);
11.  rnum--;
12.  int prej=maxj-1;             //prej查找最长递增子序列的前一个元素
13.  while(prej>=0 && rnum!=0) {
14.      if(a[prej]<a[j] && dp[prej]==rnum) {
15.          //找到最长递增子序列中一个元素a[prej]
16.          x.push_back(a[prej]);
17.          j=prej;               //从prej位置继续向前查找
18.          rnum--;
19.      }
20.      prej--;
21.  }
22.  reverse(x.begin(),x.end());   //逆置x
23.  return x;
24. }

```

55/56

优秀笔记:

int D;

vector<vector<int>> res; 1. 初始化 res 的函数, 因为 $idx \neq n-1$, 所以直接继续执行

vector<int> temp; 2. 然后 $idx+1$ 加到 temp 中, 此时 temp 变为 {1}

int tmp; 3. 然后 tmp 加进 temp, $dps(idx+1)$ pop of (1)

void dps(int idx) 4. 在 $dps()$ 中同时的 temp, 将 tmp 加到 temp, 变为 {1, 2}

if (idx == n) 5. 然后调用 $dps(idx+1)$, pop of (1)

{ res.push_back(temp); 6. 在 $dps()$ 中将 tmp 加到 temp, 此时 temp 变为 {1, 2, 3}

for (auto t: temp) 7. 将 tmp 加到 temp 中, 然后输出 {1, 2, 3}

cout << t; 8. 返回到主函数的 $dps()$, 此时 temp 为 {1, 2}, 然后对 temp 最后一个元素

return; 变为 1, 9. 然后调用 $dps(idx+1)$ pop of (1)

} 10. 在 $dps()$ 中得到了 tmp 加到 temp 中, 此时 temp 变为 {1, 3}

temp.push_back(idx); 11. 由于 $idx = n-1$, 将 $temp$ 加到 res 中, 然后输出 {1, 3}

dps(idx+1); 12. 返回到主函数的 $dps()$, 此时 temp 为 {1}, 然后对 temp 中最后一个元素

temp.pop_back(); 13. 返回到主函数的 $dps()$, 此时 temp 为空, 然后

dps(idx+1); 14. 返回到主函数的 $dps()$, 此时 temp 为 {2}

} 15. 返回到主函数的 $dps()$, 此时 temp 为 {1, 3}, 16. 由于 $idx = n-1$, 将 temp

signed main() { 17. 调用 $dps()$, 然后输出 {1, 3}

cin >> n; 18. 返回到主函数的 $dps()$, 此时 temp 为 {1, 3}, 然后对 temp 最后一个元素

dps(0); 19. 返回到主函数的 $dps()$, 此时 temp 为空, 然后结束函数并返回

}

对于 $n=1$ 的集合 $dps()$ 结束, 对每个 n 都有两种方案, 所以会分两个不同的

分为 $n-1$ 层, 而 $n-1$ 也有两种方案.....

比如 $n=1$, 要, 不要, 得到的是 {1, 2}, 都要得到的是 {1, 2}

搜索树如下

```
graph TD
    A[1] --> B[2]
    A --> C[3]
    B --> D[4]
    B --> E[5]
    C --> F[6]
    C --> G[7]
    D --> H[8]
    D --> I[9]
    E --> J[10]
    E --> K[11]
    F --> L[12]
    F --> M[13]
    G --> N[14]
    G --> O[15]
    H --> P[16]
    H --> Q[17]
    I --> R[18]
    I --> S[19]
    J --> T[20]
    J --> U[21]
    K --> V[22]
    K --> W[23]
    L --> X[24]
    L --> Y[25]
    M --> Z[26]
    M --> AA[27]
    N --> AB[28]
    N --> AC[29]
    O --> AD[30]
    O --> AE[31]
    P --> AF[32]
    P --> AG[33]
    Q --> AH[34]
    Q --> AI[35]
    R --> AJ[36]
    R --> AK[37]
    S --> AL[38]
    S --> AM[39]
    T --> AN[40]
    T --> AO[41]
    U --> AP[42]
    U --> AQ[43]
    V --> AR[44]
    V --> AS[45]
    W --> AT[46]
    W --> AU[47]
    X --> AV[48]
    X --> AW[49]
    Y --> AX[50]
    Y --> AY[51]
    Z --> AZ[52]
    Z --> BA[53]
    AA --> BB[54]
    AA --> BC[55]
    AB --> BD[56]
    AB --> BE[57]
    AC --> BF[58]
    AC --> BG[59]
    AD --> BH[60]
    AD --> BI[61]
    AE --> BJ[62]
    AE --> BK[63]
    AF --> BL[64]
    AF --> BM[65]
    AG --> BN[66]
    AG --> BO[67]
    AH --> BP[68]
    AH --> BQ[69]
    AI --> BR[70]
    AI --> BS[71]
    AJ --> BT[72]
    AJ --> BU[73]
    AK --> BV[74]
    AK --> BW[75]
    AL --> BX[76]
    AL --> BY[77]
    AM --> BZ[78]
    AM --> CA[79]
    AN --> CB[80]
    AN --> CC[81]
    AO --> CD[82]
    AO --> CE[83]
    AP --> CF[84]
    AP --> CG[85]
    AQ --> CH[86]
    AQ --> CI[87]
    AR --> CJ[88]
    AR --> CK[89]
    AS --> CL[90]
    AS --> CM[91]
    AT --> CN[92]
    AT --> CO[93]
    AU --> CP[94]
    AU --> CQ[95]
    AV --> CR[96]
    AV --> CS[97]
    AW --> CT[98]
    AW --> CU[99]
    AX --> CV[100]
    AX --> CW[101]
    AY --> CX[102]
    AY --> CY[103]
    AZ --> CZ[104]
    AZ --> DA[105]
    BA --> DB[106]
    BA --> DC[107]
    BB --> DD[108]
    BB --> DE[109]
    BC --> DF[110]
    BC --> DG[111]
    BD --> DH[112]
    BD --> DI[113]
    BE --> DJ[114]
    BE --> DK[115]
    BF --> DL[116]
    BF --> DM[117]
    BG --> DN[118]
    BG --> DO[119]
    BH --> DP[120]
    BH --> DQ[121]
    BI --> DR[122]
    BI --> DS[123]
    BJ --> DT[124]
    BJ --> DU[125]
    BK --> DV[126]
    BK --> DW[127]
    BL --> DX[128]
    BL --> DY[129]
    BM --> DZ[130]
    BM --> EA[131]
    BN --> EB[132]
    BN --> EC[133]
    BO --> ED[134]
    BO --> EE[135]
    BP --> EF[136]
    BP --> EG[137]
    BQ --> EH[138]
    BQ --> EI[139]
    BR --> EJ[140]
    BR --> EK[141]
    BS --> EL[142]
    BS --> EM[143]
    BT --> EN[144]
    BT --> EO[145]
    BU --> EP[146]
    BU --> EQ[147]
    BV --> ER[148]
    BV --> ES[149]
    BW --> ET[150]
    BW --> EU[151]
    BX --> EV[152]
    BX --> EW[153]
    BY --> EX[154]
    BY --> EY[155]
    BZ --> EZ[156]
    BZ --> FA[157]
    CA --> FB[158]
    CA --> FC[159]
    CB --> FD[160]
    CB --> FE[161]
    CC --> FF[162]
    CC --> FG[163]
    CD --> FH[164]
    CD --> FI[165]
    CE --> FJ[166]
    CE --> FK[167]
    CF --> FL[168]
    CF --> FM[169]
    CG --> FN[170]
    CG --> FO[171]
    CH --> FP[172]
    CH --> FQ[173]
    CI --> FR[174]
    CI --> FS[175]
    CJ --> FT[176]
    CJ --> FU[177]
    CK --> FV[178]
    CK --> FW[179]
    CL --> FX[180]
    CL --> FY[181]
    CM --> FZ[182]
    CM --> GA[183]
    CN --> GB[184]
    CN --> GC[185]
    CO --> GD[186]
    CO --> GE[187]
    CP --> GF[188]
    CP --> GG[189]
    CQ --> GH[190]
    CQ --> GI[191]
    CR --> GJ[192]
    CR --> GK[193]
    CS --> GL[194]
    CS --> GM[195]
    CT --> GN[196]
    CT --> GO[197]
    CU --> GP[198]
    CU --> GQ[199]
    CV --> GR[200]
    CV --> GS[201]
    CW --> GT[202]
    CW --> GU[203]
    CX --> GV[204]
    CX --> GW[205]
    CY --> GX[206]
    CY --> GY[207]
    CZ --> GZ[208]
    CZ --> HA[209]
    DA --> HB[210]
    DA --> HC[211]
    DB --> HD[212]
    DB --> HE[213]
    DC --> HF[214]
    DC --> HG[215]
    DD --> HH[216]
    DD --> HI[217]
    DE --> HJ[218]
    DE --> HK[219]
    DF --> HL[220]
    DF --> HM[221]
    DG --> HN[222]
    DG --> HO[223]
    DH --> HP[224]
    DH --> HQ[225]
    DI --> HR[226]
    DI --> HS[227]
    DJ --> HT[228]
    DJ --> HU[229]
    DK --> HV[230]
    DK --> HW[231]
    DL --> HX[232]
    DL --> HY[233]
    DM --> HZ[234]
    DM --> IA[235]
    DN --> IB[236]
    DN --> IC[237]
    DO --> ID[238]
    DO --> IE[239]
    DP --> IF[240]
    DP --> IG[241]
    DQ --> IH[242]
    DQ --> II[243]
    DR --> IJ[244]
    DR --> IK[245]
    DS --> IL[246]
    DS --> IM[247]
    DT --> IN[248]
    DT --> IO[249]
    DU --> IP[250]
    DU --> IQ[251]
    DV --> IR[252]
    DV --> IS[253]
    DW --> IT[2
```

本例的回溯：因为随着新的约束条件不断的添加

如果回溯：我发现在这条路所走的状态，比前解劣影响，那我就把这条路在搜索树中是不需要再走一遍，即节省搜索时间

回溯就是必须从当前所访问的父节点回退，并且这个节点可能被删除，即回溯不受前解影响，所以叫回溯。

第5章 回溯法——理论算法1

5.1.2 n 皇后问题

5.1.1 问题的回溯算法

· 一个要解决的问题的解通常是由若干个小的决策步骤组成的决策树，其树可以表示成解树

· 解树中 $X = \{x_1, x_2, \dots, x_n\}$ ，其中分量 x_i 对第 i 个决策

· 决策可以有多个或两个可行值，表示为 $X_i \in S_i$ ， S_i 为 x_i 的可行值集合

· X 中分量 x_i 所有的可行值组合不构成问题的解树分量空间，解树称为解空间

或者解空间树

解空间树图例

根节点 $X_0 = \{x_0\}$

第一层节点 $X_1 = \{x_1\}$

第二层节点 $X_2 = \{x_2\}$

第三层节点 $X_3 = \{x_3\}$

第四层节点 $X_4 = \{x_4\}$

第五层节点 $X_5 = \{x_5\}$

第六层节点 $X_6 = \{x_6\}$

第七层节点 $X_7 = \{x_7\}$

第八层节点 $X_8 = \{x_8\}$

第九层节点 $X_9 = \{x_9\}$

第十层节点 $X_{10} = \{x_{10}\}$

第十一层节点 $X_{11} = \{x_{11}\}$

第十二层节点 $X_{12} = \{x_{12}\}$

第十三层节点 $X_{13} = \{x_{13}\}$

第十四层节点 $X_{14} = \{x_{14}\}$

第十五层节点 $X_{15} = \{x_{15}\}$

第十六层节点 $X_{16} = \{x_{16}\}$

第十七层节点 $X_{17} = \{x_{17}\}$

第十八层节点 $X_{18} = \{x_{18}\}$

第十九层节点 $X_{19} = \{x_{19}\}$

第二十层节点 $X_{20} = \{x_{20}\}$

第二十一层节点 $X_{21} = \{x_{21}\}$

第二十二层节点 $X_{22} = \{x_{22}\}$

第二十三层节点 $X_{23} = \{x_{23}\}$

第二十四层节点 $X_{24} = \{x_{24}\}$

第二十五层节点 $X_{25} = \{x_{25}\}$

第二十六层节点 $X_{26} = \{x_{26}\}$

第二十七层节点 $X_{27} = \{x_{27}\}$

第二十八层节点 $X_{28} = \{x_{28}\}$

第二十九层节点 $X_{29} = \{x_{29}\}$

第三十层节点 $X_{30} = \{x_{30}\}$

第三十一层节点 $X_{31} = \{x_{31}\}$

第三十二层节点 $X_{32} = \{x_{32}\}$

第三十三层节点 $X_{33} = \{x_{33}\}$

第三十四层节点 $X_{34} = \{x_{34}\}$

第三十五层节点 $X_{35} = \{x_{35}\}$

第三十六层节点 $X_{36} = \{x_{36}\}$

第三十七层节点 $X_{37} = \{x_{37}\}$

第三十八层节点 $X_{38} = \{x_{38}\}$

第三十九层节点 $X_{39} = \{x_{39}\}$

第四十层节点 $X_{40} = \{x_{40}\}$

第四十一层节点 $X_{41} = \{x_{41}\}$

第四十二层节点 $X_{42} = \{x_{42}\}$

第四十三层节点 $X_{43} = \{x_{43}\}$

第四十四层节点 $X_{44} = \{x_{44}\}$

第四十五层节点 $X_{45} = \{x_{45}\}$

第四十六层节点 $X_{46} = \{x_{46}\}$

第四十七层节点 $X_{47} = \{x_{47}\}$

第四十八层节点 $X_{48} = \{x_{48}\}$

第四十九层节点 $X_{49} = \{x_{49}\}$

第五十层节点 $X_{50} = \{x_{50}\}$

第五十一层节点 $X_{51} = \{x_{51}\}$

第五十二层节点 $X_{52} = \{x_{52}\}$

第五十三层节点 $X_{53} = \{x_{53}\}$

第五十四层节点 $X_{54} = \{x_{54}\}$

第五十五层节点 $X_{55} = \{x_{55}\}$

第五十六层节点 $X_{56} = \{x_{56}\}$

第五十七层节点 $X_{57} = \{x_{57}\}$

第五十八层节点 $X_{58} = \{x_{58}\}$

第五十九层节点 $X_{59} = \{x_{59}\}$

第六十层节点 $X_{60} = \{x_{60}\}$

第六十一层节点 $X_{61} = \{x_{61}\}$

第六十二层节点 $X_{62} = \{x_{62}\}$

第六十三层节点 $X_{63} = \{x_{63}\}$

第六十四层节点 $X_{64} = \{x_{64}\}$

第六十五层节点 $X_{65} = \{x_{65}\}$

第六十六层节点 $X_{66} = \{x_{66}\}$

第六十七层节点 $X_{67} = \{x_{67}\}$

第六十八层节点 $X_{68} = \{x_{68}\}$

第六十九层节点 $X_{69} = \{x_{69}\}$

第七十层节点 $X_{70} = \{x_{70}\}$

第七十一层节点 $X_{71} = \{x_{71}\}$

第七十二层节点 $X_{72} = \{x_{72}\}$

第七十三层节点 $X_{73} = \{x_{73}\}$

第七十四层节点 $X_{74} = \{x_{74}\}$

第七十五层节点 $X_{75} = \{x_{75}\}$

第七十六层节点 $X_{76} = \{x_{76}\}$

第七十七层节点 $X_{77} = \{x_{77}\}$

第七十八层节点 $X_{78} = \{x_{78}\}$

第七十九层节点 $X_{79} = \{x_{79}\}$

第八十层节点 $X_{80} = \{x_{80}\}$

第八十一层节点 $X_{81} = \{x_{81}\}$

第八十二层节点 $X_{82} = \{x_{82}\}$

第八十三层节点 $X_{83} = \{x_{83}\}$

第八十四层节点 $X_{84} = \{x_{84}\}$

第八十五层节点 $X_{85} = \{x_{85}\}$

第八十六层节点 $X_{86} = \{x_{86}\}$

第八十七层节点 $X_{87} = \{x_{87}\}$

第八十八层节点 $X_{88} = \{x_{88}\}$

第八十九层节点 $X_{89} = \{x_{89}\}$

第九十层节点 $X_{90} = \{x_{90}\}$

第九十一层节点 $X_{91} = \{x_{91}\}$

第九十二层节点 $X_{92} = \{x_{92}\}$

第九十三层节点 $X_{93} = \{x_{93}\}$

第九十四层节点 $X_{94} = \{x_{94}\}$

第九十五层节点 $X_{95} = \{x_{95}\}$

第九十六层节点 $X_{96} = \{x_{96}\}$

第九十七层节点 $X_{97} = \{x_{97}\}$

第九十八层节点 $X_{98} = \{x_{98}\}$

第九十九层节点 $X_{99} = \{x_{99}\}$

第一百层节点 $X_{100} = \{x_{100}\}$

第一百零一层节点 $X_{101} = \{x_{101}\}$

第一百零二层节点 $X_{102} = \{x_{102}\}$

第一百零三层节点 $X_{103} = \{x_{103}\}$

第一百零四层节点 $X_{104} = \{x_{104}\}$

第一百零五层节点 $X_{105} = \{x_{105}\}$

第一百零六层节点 $X_{106} = \{x_{106}\}$

第一百零七层节点 $X_{107} = \{x_{107}\}$

第一百零八层节点 $X_{108} = \{x_{108}\}$

第一百零九层节点 $X_{109} = \{x_{109}\}$

第一百一十层节点 $X_{110} = \{x_{110}\}$

第一百一十一层节点 $X_{111} = \{x_{111}\}$

第一百一十二层节点 $X_{112} = \{x_{112}\}$

第一百一十三层节点 $X_{113} = \{x_{113}\}$

第一百一十四层节点 $X_{114} = \{x_{114}\}$

第一百一十五层节点 $X_{115} = \{x_{115}\}$

第一百一十六层节点 $X_{116} = \{x_{116}\}$

第一百一十七层节点 $X_{117} = \{x_{117}\}$

第一百一十八层节点 $X_{118} = \{x_{118}\}$

第一百一十九层节点 $X_{119} = \{x_{119}\}$

第一百二十层节点 $X_{120} = \{x_{120}\}$

第一百二十一层节点 $X_{121} = \{x_{121}\}$

第一百二十二层节点 $X_{122} = \{x_{122}\}$

第一百二十三层节点 $X_{123} = \{x_{123}\}$

第一百二十四层节点 $X_{124} = \{x_{124}\}$

第一百二十五层节点 $X_{125} = \{x_{125}\}$

第一百二十六层节点 $X_{126} = \{x_{126}\}$

第一百二十七层节点 $X_{127} = \{x_{127}\}$

第一百二十八层节点 $X_{128} = \{x_{128}\}$

第一百二十九层节点 $X_{129} = \{x_{129}\}$

第一百三十层节点 $X_{130} = \{x_{130}\}$

第一百三十一层节点 $X_{131} = \{x_{131}\}$

第一百三十二层节点 $X_{132} = \{x_{132}\}$

第一百三十三层节点 $X_{133} = \{x_{133}\}$

第一百三十四层节点 $X_{134} = \{x_{134}\}$

第一百三十五层节点 $X_{135} = \{x_{135}\}$

第一百三十六层节点 $X_{136} = \{x_{136}\}$

第一百三十七层节点 $X_{137} = \{x_{137}\}$

第一百三十八层节点 $X_{138} = \{x_{138}\}$

第一百三十九层节点 $X_{139} = \{x_{139}\}$

第一百四十层节点 $X_{140} = \{x_{140}\}$

第一百四十一层节点 $X_{141} = \{x_{141}\}$

第一百四十二层节点 $X_{142} = \{x_{142}\}$

1. **C++ vector 操作**

vector<int> a(b); // 用向量b的向量a赋值, a的值全等于b的值
#include <vector>
vector<int> a(b);
a.assign(b.begin(), b.begin()+5); // b为向量, 将b的0-2个元素赋值给a

2. **邻接矩阵**

0	5	∞	1	∞
5	0	∞	∞	∞
∞	∞	0	2	3
1	∞	2	0	2
∞	∞	3	2	0

3. **DFS 模板**

```
const int INF = 0x3f3f3f3f;  
int n, s;  
vector<vector<int>>> A = {{0, 5, INF, 1, INF}, {5, 0, INF, INF, INF},  
{{INF, INF, 0, 2, 3}, {1, INF, 2, 0, 2}, {INF, INF, 3, 2, 0}}};  
vector<vector<int>>> ans; // 自拟答案  
vector<int> x; // 自拟答案  
vector<int> visited  
void dfs(int i, int curLen, int r) // 回溯算法  
if (i == r) // 记录答案  
ans.push_back(x); // 将x加入到ans中  
ans.back().push_back(curLen); // 防止未遍历到的情况  
}  
else if
```

4. **DFS 模板**

记笔记的好处：1.头脑并用，防止静态（打盹）；2.扩大大脑外存，记住历史；3.形成习惯（学习、工作、生活）；4.练字（磨练意志），字如其人。